Cross-Domain Numerical Consistency Checks for Baker's Geometric Constant π_g

Wayne Baker

August 27, 2026
Referee-safe computational appendix, Version 2.2.0

Abstract

This report documents a synchronized, reproducible numerical verification suite for Baker's algebraically constructed geometric constant π_g. The construction module begins with the exact hexagon seed $c_0 = 1/2$, selects the required cubic branch by a deterministic sign-changing bracket, forms the polygon sequence $P_L = 6 \cdot 3^L c_L$, and applies the scale-3 even-power Richardson table. The classical constant π is not consulted during construction or result selection. After π_g has been fixed, nine cross-domain checks are performed using direct residuals, explicit truncation enclosures, or two-sided inequalities. All nine checks pass at 120-decimal-digit working precision. These computations are consistency evidence, not a substitute for a proof of $\pi_g = \pi$.

Contents

1 Status and claim boundary **2**

2 Oracle-free algebraic construction **2**

2.1 Unique cubic branch 2

2.2 Polygon sequence and N-series acceleration 3

3 Verification protocol **4**

3.1 Numerical environment 4

3.2 Acceptance criteria 4

3.3 Bounded checks 5

4 Results **6**

5 Reproducibility and provenance **7**

6 Conclusion **7**

A Oracle-free construction script **7**

B Verification script **11**

C Full output log **20**

1 Status and claim boundary

The computational claim is deliberately limited:

> The algebraic pipeline produces a fixed high-precision value π_g. That value satisfies nine stated numerical consistency checks within their declared acceptance bounds.

The suite does *not* prove $\pi_g = \pi$. Equality must rest on the associated mathematical argument connecting the chord cascade and polygonal limit to the classical analytic constant. The tests below illustrate consequences of that equality and provide a reproducibility audit of the implementation.

The phrase "independent identity verification" is avoided. In particular, the `mpmath` implementations of special functions may internally use classical π. Those tests are therefore cross-domain library-level consistency checks, not algorithmically independent derivations of π.

2 Oracle-free algebraic construction

2.1 Unique cubic branch

Let

$$f_c(u) = 4u^3 - 3u + c, \qquad 0 < c \le \frac{1}{2}.$$

The trisection step requires the small positive root. It is selected without an initial guess and without reference to π.

Proposition 1. *For every $0 < c \le 1/2$, the interval $(0, c/2)$ contains exactly one zero of f_c.*

Proof. At the endpoints,

$$f_c(0) = c > 0$$

and

$$f_c\left(\frac{c}{2}\right) = 4\left(\frac{c}{2}\right)^3 - 3\left(\frac{c}{2}\right) + c = \frac{c(c^2-1)}{2} < 0.$$

Moreover,

$$f_c'(u) = 12u^2 - 3 < 0$$

for $0 \le u \le c/2 \le 1/4$. Thus f_c is strictly decreasing on the bracket, and the sign change contains exactly one root. □

The implementation bisects this certified branch until the bracket is smaller than the prescribed guard scale. The maximum final bracket width in the present run was

`9.96319669471211514494456752277843452094e-101`,

and the maximum midpoint polynomial residual was

`1.40735005136297493732277918456015428347 8e-100`.

These are floating-point diagnostics attached to an exact branch-selection argument.

2.2 Polygon sequence and N-series acceleration

Starting from

$$c_0 = \frac{1}{2},$$

the recurrence chooses c_{L+1} as the unique root in $(0, c_L/2)$ of

$$4c_{L+1}^3 - 3c_{L+1} + c_L = 0.$$

The polygon quantity is

$$P_L = 6 \cdot 3^L c_L.$$

The present computation uses 11 inputs,

$$L = 0, 1, \ldots, 10.$$

If a sequence has an even inverse-power expansion

$$P_N = Q + \frac{a_1}{N^2} + \frac{a_2}{N^4} + \cdots,$$

then replacing N by $3N$ scales the stage-s leading error by 9^{-s}. The triangular extrapolation is therefore

$$T_0(L) = P_L,$$
$$T_s(L) = T_{s-1}(L) + \frac{T_{s-1}(L) - T_{s-1}(L-1)}{9^s - 1}.$$

The selected value is deterministically the final table entry $T_{10}(10)$. No comparison with classical π is used to select a level or candidate.

L	$N = 6 \cdot 3^L$	P_L	Richardson diagonal $T_L(L)$
0	6	3.0	3.0
1	18	3.125667198004746279331	3.141375597755339564247
2	54	3.139820761165694741092	3.141592636045881720495
3	162	3.141395747035548337671	3.141592653589701467835
4	486	3.141570774718079713566	3.141592653589793238428
5	1458	3.141590222599532605478	3.141592653589793238463
6	4374	3.141592383479708542326	3.141592653589793238463
7	13122	3.141592623577560917448	3.141592653589793238463
8	39366	3.141592650255100749855	3.141592653589793238463
9	118098	3.141592653219271850735	3.141592653589793238463
10	354294	3.141592653548624195 38	3.141592653589793238463

The resulting value is

$$\pi_g = \frac{3.14159265358979323846264338327950288419716939937 5}{10582097494459230781640628620899600343013114606653 5}.$$

The last two extrapolation diagonal entries differ by

$$1.68888783015368936452420728674938486347 5e-68.$$

This difference is an internal convergence indicator, not a rigorous error bound.

Only after π_g was fixed was it compared with the library reference value

$$\pi = \frac{\texttt{3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 8}}{}.$$

The diagnostic difference was

$$|\pi_g - \pi| = \texttt{2.624604694196050533234281235447495468803e-81}.$$

This comparison is reported transparently but is not part of the construction or any candidate-selection rule.

3 Verification protocol

3.1 Numerical environment

- Python: `3.13.5`.
- `mpmath`: `1.3.0`.
- Working precision: 120 decimal digits.
- Construction inputs: 11 polygon levels, $L = 0, \ldots, 10$.
- Fixed threshold for direct special-function residuals: 10^{-65}.
- Every series/product check uses either an explicit remainder bound or a finite two-sided enclosure.

No printed value "0" is interpreted as exact equality. A zero metric means only that the computed violation was nonpositive at the active precision or that a target lay inside a finite enclosure.

3.2 Acceptance criteria

ID	Method	Acceptance bound
G1	finite quadrature, analytic two-sided tail bound, repeat-precision stability check	`3.09235401e-75`
B2	central-binomial zeta(2) series with geometric ratio tail bound	`7.54477984e-76`
L3	Euler-transformed alternating series with explicit ratio tail bound	`3.38123188e-71`
W4	standard two-sided finite Wallis enclosure	`1.0e-105`
S5	exact log-factorial sum tested inside Robbins' two-sided bounds	`1.0e-105`
H6	direct residual gamma(1/2)^2 - pi_g	`1.0e-65`
E7	direct complex residual exp(i*pi_g)+1	`1.0e-65`
R8	direct normalized residual at three rational arguments	`1.0e-65`

ID	Method	Acceptance bound
N9	closed-form sine Taylor remainder bound combined with the tableau's fixed data-independent linear weights and the propagated cubic-root bracket width	`1.24578953e-77`

3.3 Bounded checks

Gaussian integral. The finite integral is evaluated on $[-A, A]$ with $A = 13$. The omitted two-sided tail satisfies

$$2\int_A^\infty e^{-x^2}\,dx \le \frac{e^{-A^2}}{A}.$$

The acceptance allowance is this analytic tail bound plus a repeat-precision quadrature stability term and a small rounding reserve.

Basel identity. The suite uses the rapidly convergent positive series

$$\zeta(2) = 3\sum_{n=1}^{\infty} \frac{1}{n^2\binom{2n}{n}}.$$

For the unscaled term $a_n = 1/(n^2\binom{2n}{n})$,

$$\frac{a_{n+1}}{a_n} = \frac{n^2}{(2n+2)(2n+1)} < \frac{1}{4}.$$

Consequently, after N terms the omitted $\zeta(2)$ tail is smaller than $4a_{N+1}$.

Gregory-Leibniz identity. Euler transformation is applied to

$$\frac{\pi}{4} = \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}.$$

Writing

$$I_k = \int_0^1 (1-x^2)^k\,dx,$$

gives

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{I_k}{2^{k+1}}, \qquad I_0 = 1, \qquad I_{k+1} = \frac{2(k+1)}{2k+3} I_k.$$

The transformed term ratio is

$$\frac{t_{k+1}}{t_k} = \frac{k+1}{2k+3} < \frac{1}{2},$$

so the omitted tail is smaller than twice the first omitted term.

Wallis product. For

$$W_N = \prod_{n=1}^{N} \frac{(2n)^2}{(2n-1)(2n+1)},$$

the standard finite Wallis enclosure is

$$W_N < \frac{\pi}{2} < W_N \frac{2N+1}{2N}.$$

The test passes only when $\pi_g/2$ lies inside this interval.

Robbins-Stirling bounds. For each tested integer n, the suite checks the exact logarithmic factorial sum against Robbins' bounds

$$\sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{1/(12n+1)} < n! < \sqrt{2\pi n}\left(\frac{n}{e}\right)^n e^{1/(12n)}.$$

The implementation replaces π by the already constructed π_g and verifies inclusion for $n = 10, 100, 1000, 10000$.

4 Results

ID	Check	Observed metric	Status
G1	Gaussian integral	`3.08328432e-75`	PASS
B2	Basel identity	`7.51408053e-76`	PASS
L3	Gregory-Leibniz identity	`3.37397563e-71`	PASS
W4	Wallis product	`0`	PASS
S5	Robbins-Stirling bounds	`0`	PASS
H6	Gamma at one half	`2.62460469e-81`	PASS
E7	Euler identity	`2.62460469e-81`	PASS
R8	Euler reflection formula	`4.94323317e-82`	PASS
N9	Richardson tableau a priori enclosure	`2.62460469e-81`	PASS

All nine declared checks passed:

`PI_G_CROSS_DOMAIN_CONSISTENCY_CHECKS_PASS`.

The strongest interpretation supported by the computation is:

> At 120-decimal-digit working precision, the deterministically constructed value π_g is consistent with the listed bounded series, product, asymptotic, analytic, and special-function relations under the stated acceptance rules.

The computation does not establish functional identity "across all domains," nor does it replace a theorem proving $\pi_g = \pi$.

5 Reproducibility and provenance

The construction and verification are physically separated:

- `pi_g_construction.py` contains the algebraic seed, certified branch selection, polygon sequence, and extrapolation. It does not import or reference classical π.
- `verify_pi_g_identities.py` imports the completed construction and then performs the reference comparison and nine checks.

The canonical run command is

```
python verify_pi_g_identities.py --output-dir output
```

The report is regenerated with

```
python build_pi_g_report.py --compile
```

The output manifest records software metadata and SHA-256 hashes for both scripts and the principal numerical ledgers.

6 Conclusion

The revised suite removes the two central weaknesses of the earlier version. First, the cubic branch is selected by a proved sign-changing interval rather than a heuristic Newton initial guess. Second, PASS decisions are based on direct residuals or explicit enclosures, not on the near-tautological comparison of a π_g-based expression with the same expression evaluated using classical π.

The resulting package is suitable as a computational appendix accompanying a mathematical proof of $\pi_g = \pi$. Its correct status is *empirical and reproducible corroboration.*

References

[1] H. Robbins, "A Remark on Stirling's Formula," *American Mathematical Monthly* 62 (1955), 26–29.

[2] J. M. Borwein and P. B. Borwein, *Pi and the AGM*, Wiley, 1987.

[3] L. F. Richardson, "The Deferred Approach to the Limit," *Philosophical Transactions of the Royal Society A* 226 (1927), 299–361.

A Oracle-free construction script

Listing 1: `pi_g_construction.py`

```
1 #!/usr/bin/env python3
2 """Oracle-free construction kernel for Baker's geometric constant pi_g.
3
```

```
This module deliberately does not import or reference the classical constant pi.
It starts from the algebraic hexagon seed c_0 = 1/2, repeatedly selects the
unique small positive root of

    4 u^3 - 3 u + c = 0,

and applies the N-series Richardson table with scale factor 3.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Sequence

from mpmath import mp


@dataclass(frozen=True)
class RootCertificate:
    """Numerical record for one deterministically bracketed cubic root."""

    value: mp.mpf
    lower: mp.mpf
    upper: mp.mpf
    width: mp.mpf
    iterations: int
    polynomial_residual: mp.mpf


@dataclass(frozen=True)
class PiGConstructionResult:
    """Complete output of the algebraic polygon/extrapolation pipeline."""

    value: mp.mpf
    level_count: int
    maximum_level: int
    work_dps: int
    polygon_values: tuple[mp.mpf, ...]
    extrapolated_diagonal: tuple[mp.mpf, ...]
    internal_last_diagonal_delta: mp.mpf
    maximum_root_bracket_width: mp.mpf
    maximum_root_polynomial_residual: mp.mpf
    root_iterations: tuple[int, ...]


def _cubic(u: mp.mpf, c: mp.mpf) -> mp.mpf:
    return 4 * u**3 - 3 * u + c


def small_positive_trisection_root(c: mp.mpf) -> RootCertificate:
    """Return the unique root in (0, c/2) by deterministic bisection.

    For 0 < c <= 1/2,

        f_c(0) = c > 0,
        f_c(c/2) = c(c^2 - 1)/2 < 0,

    and f'_c(u) = 12u^2 - 3 < 0 on [0, c/2].  Hence this interval
    contains exactly one root, which is the required small positive branch.

    The mathematical branch selection is exact.  The returned decimal value
    is a high-precision midpoint of the final sign-changing bracket.
    """

    c = mp.mpf(c)
    if not (mp.mpf("0") < c <= mp.mpf("0.5")):
        raise ValueError(f"Expected 0 < c <= 1/2, received {c!r}")

```

```
    lower = mp.mpf("0")
    upper = c / 2
    f_lower = _cubic(lower, c)
    f_upper = _cubic(upper, c)

    if not (f_lower > 0 and f_upper < 0):
        raise ArithmeticError("Initial cubic bracket does not change sign")

    # Preserve roughly 20 decimal guard digits inside the active mp context.
    target_width = mp.power(10, -(mp.dps - 20))
    maximum_iterations = max(256, 8 * mp.dps)

    iterations = 0
    for iterations in range(1, maximum_iterations + 1):
        midpoint = (lower + upper) / 2
        if midpoint == lower or midpoint == upper:
            break

        f_midpoint = _cubic(midpoint, c)
        if f_midpoint > 0:
            lower = midpoint
        elif f_midpoint < 0:
            upper = midpoint
        else:
            lower = upper = midpoint
            break

        if upper - lower <= target_width:
            break
    else:
        raise ArithmeticError("Bisection did not reach the requested bracket width")

    value = (lower + upper) / 2
    width = upper - lower
    residual = abs(_cubic(value, c))

    if width > target_width * 2:
        raise ArithmeticError(
            f"Final root bracket is wider than expected: {mp.nstr(width, 12)}"
        )

    return RootCertificate(
        value=value,
        lower=lower,
        upper=upper,
        width=width,
        iterations=iterations,
        polynomial_residual=residual,
    )


def richardson_n_series(values: Sequence[mp.mpf]) -> tuple[mp.mpf, tuple[mp.mpf, ...]]:
    """Apply the scale-3 even-power Richardson/N-series table.

    If the polygon error has an expansion in N^{-2}, N^{-4}, ..., replacing
    N by 3N multiplies the leading term at stage s by 9^{-s}.  Therefore the
    stage-s denominator is 9^s - 1.
    """

    if not values:
        raise ValueError("At least one polygon value is required")

    n = len(values)
    table: list[list[mp.mpf | None]] = [[mp.mpf(v) for v in values]]

    for stage in range(1, n):
        previous = table[-1]
        current: list[mp.mpf | None] = [None] * n
```

```
        denominator = mp.power(9, stage) - 1
        for column in range(stage, n):
            right = previous[column]
            left = previous[column - 1]
            if right is None or left is None:
                raise ArithmeticError("Malformed Richardson table")
            current[column] = right + (right - left) / denominator
        table.append(current)

    diagonal: list[mp.mpf] = []
    for stage in range(n):
        value = table[stage][stage]
        if value is None:
            raise ArithmeticError("Missing Richardson diagonal entry")
        diagonal.append(value)

    final = table[-1][-1]
    if final is None:
        raise ArithmeticError("Missing final Richardson value")

    return final, tuple(diagonal)


def richardson_tableau_weights(level_count: int, work_dps: int) -> tuple[mp.mpf, ...]:
    """Return the fixed, data-independent weights of the final diagonal entry.

    ``richardson_n_series`` is linear in its input values, so the final
    diagonal entry can be written as ``sum_i weight_i * values[i]`` for a
    weight vector that depends only on ``level_count`` (not on the data).
    Those weights are recovered here by feeding unit basis vectors through
    the identical routine used for construction.  This is exact linear
    algebra, not a numerical fit: no reference constant is consulted.
    """

    with mp.workdps(work_dps):
        weights = []
        for i in range(level_count):
            basis = [mp.mpf(1) if j == i else mp.mpf(0) for j in range(level_count)]
            final, _ = richardson_n_series(basis)
            weights.append(+final)
        return tuple(weights)


def compute_pi_g(*, level_count: int = 11, work_dps: int = 120) -> PiGConstructionResult:
    """Compute pi_g without consulting the classical constant pi.

    ``level_count=11`` means the polygon inputs L=0,...,10 are used.  The
    construction result is selected deterministically as the final entry of
    the corresponding Richardson table; no reference constant is consulted.
    """

    if level_count < 2:
        raise ValueError("level_count must be at least 2")
    if work_dps < 50:
        raise ValueError("work_dps must be at least 50")

    with mp.workdps(work_dps):
        c = mp.mpf(1) / 2
        polygon_values: list[mp.mpf] = []
        root_certificates: list[RootCertificate] = []

        for level in range(level_count):
            n_sides = mp.mpf(6) * mp.power(3, level)
            polygon_values.append(n_sides * c)

            if level + 1 < level_count:
                certificate = small_positive_trisection_root(c)
                root_certificates.append(certificate)
```

```
                c = certificate.value

        final, diagonal = richardson_n_series(polygon_values)
        internal_delta = abs(diagonal[-1] - diagonal[-2])

        maximum_width = max(cert.width for cert in root_certificates)
        maximum_residual = max(cert.polynomial_residual for cert in root_certificates)
        iterations = tuple(cert.iterations for cert in root_certificates)

        return PiGConstructionResult(
            value=+final,
            level_count=level_count,
            maximum_level=level_count - 1,
            work_dps=work_dps,
            polygon_values=tuple(+v for v in polygon_values),
            extrapolated_diagonal=tuple(+v for v in diagonal),
            internal_last_diagonal_delta=+internal_delta,
            maximum_root_bracket_width=+maximum_width,
            maximum_root_polynomial_residual=+maximum_residual,
            root_iterations=iterations,
        )
```

B Verification script

Listing 2: `verify_pi_g_identities.py`

```
#!/usr/bin/env python3
"""Referee-safe cross-domain numerical consistency checks for Baker's pi_g.

The algebraic construction is imported from ``pi_g_construction.py``.  The
classical constant pi is used only after construction, for a separately
labelled reference comparison.  It is never used to choose or alter pi_g.

The nine PASS criteria are direct identity residuals or explicit finite
truncation enclosures.  They are not comparisons between two right-hand sides
that both already contain pi.
"""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import math
import platform
import sys
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Callable, Iterable

import mpmath
from mpmath import mp

from pi_g_construction import PiGConstructionResult, compute_pi_g, richardson_tableau_weights


DEFAULT_WORK_DPS = 120
DEFAULT_LEVEL_COUNT = 11
DIRECT_RESIDUAL_TOLERANCE_TEXT = "1e-65"


@dataclass(frozen=True)
class CheckResult:
```

```
    identifier: str
    name: str
    domain: str
    method: str
    observed_metric_name: str
    observed_metric: mp.mpf
    acceptance_bound: mp.mpf
    passed: bool
    details: str


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def sci(value: mp.mpf, digits: int = 18) -> str:
    value = mp.mpf(value)
    if value == 0:
        return "0"
    return mp.nstr(value, digits, min_fixed=0, max_fixed=0)


def dec(value: mp.mpf, digits: int = 80) -> str:
    return mp.nstr(mp.mpf(value), digits)


def check_gaussian_integral(pi_g: mp.mpf, work_dps: int) -> CheckResult:
    """Check 2*integral_0^A exp(-x^2) dx against sqrt(pi_g).

    The omitted two-sided tail obeys

        2*integral_A^infinity exp(-x^2) dx <= exp(-A^2)/A.

    A repeat-precision difference is added as a conservative numerical
    stability allowance for the finite quadrature.
    """

    cutoff = mp.mpf(13)

    with mp.workdps(work_dps):
        integral_low = 2 * mp.quad(lambda x: mp.exp(-(x * x)), [0, cutoff])
    with mp.workdps(work_dps + 30):
        integral_high = 2 * mp.quad(lambda x: mp.exp(-(x * x)), [0, cutoff])

    integral_high = +integral_high
    stability = abs(integral_high - integral_low)
    tail_bound = mp.exp(-(cutoff * cutoff)) / cutoff
    roundoff = mp.power(10, -(work_dps - 15))
    allowance = tail_bound + 10 * stability + roundoff
    residual = abs(mp.sqrt(pi_g) - integral_high)

    return CheckResult(
        identifier="G1",
        name="Gaussian integral",
        domain="Analysis / probability",
        method="finite quadrature, analytic two-sided tail bound, repeat-precision stability check",
        observed_metric_name="abs(sqrt(pi_g) - finite integral)",
        observed_metric=residual,
        acceptance_bound=allowance,
        passed=residual <= allowance,
        details=(
            f"cutoff A=13; analytic tail<={sci(tail_bound)}; "
            f"repeat-precision delta={sci(stability)}"
        ),
```

```
    )


def check_basel(pi_g: mp.mpf) -> CheckResult:
    """Check zeta(2)=pi_g^2/6 using a rapidly convergent positive series.

        zeta(2) = 3 * sum_{n>=1} 1/(n^2 * binom(2n,n)).

    The term ratio is n^2/((2n+2)(2n+1)) < 1/4, so the omitted zeta tail
    after n=N is smaller than 4 times the first omitted unscaled term.
    """

    terms = 120
    term = mp.mpf(1) / 2  # n=1
    partial_unscaled = mp.mpf(0)

    for n in range(1, terms + 1):
        partial_unscaled += term
        term *= mp.mpf(n * n) / ((2 * n + 2) * (2 * n + 1))

    partial = 3 * partial_unscaled
    tail_bound = 4 * term
    roundoff = mp.power(10, -(mp.dps - 15))
    residual = abs((pi_g * pi_g) / 6 - partial)
    allowance = tail_bound + roundoff

    return CheckResult(
        identifier="B2",
        name="Basel identity",
        domain="Number theory",
        method="central-binomial zeta(2) series with geometric ratio tail bound",
        observed_metric_name="abs(pi_g^2/6 - accelerated partial sum)",
        observed_metric=residual,
        acceptance_bound=allowance,
        passed=residual <= allowance,
        details=f"terms={terms}; explicit tail<={sci(tail_bound)}",
    )


def check_leibniz(pi_g: mp.mpf) -> CheckResult:
    """Check pi_g/4 using the Euler-transformed Gregory-Leibniz series.

    If I_0=1 and I_{k+1}=2(k+1)I_k/(2k+3), then

        pi/4 = sum_{k>=0} I_k/2^(k+1).

    Consecutive transformed terms have ratio (k+1)/(2k+3)<1/2, so the
    omitted tail is smaller than twice the first omitted term.
    """

    terms = 230
    term = mp.mpf(1) / 2
    partial = mp.mpf(0)

    for k in range(terms):
        partial += term
        term *= mp.mpf(k + 1) / (2 * k + 3)

    tail_bound = 2 * term
    roundoff = mp.power(10, -(mp.dps - 15))
    residual = abs(pi_g / 4 - partial)
    allowance = tail_bound + roundoff

    return CheckResult(
        identifier="L3",
        name="Gregory-Leibniz identity",
        domain="Series",
        method="Euler-transformed alternating series with explicit ratio tail bound",
```

```
        observed_metric_name="abs(pi_g/4 - transformed partial sum)",
        observed_metric=residual,
        acceptance_bound=allowance,
        passed=residual <= allowance,
        details=f"transformed terms={terms}; explicit tail<={sci(tail_bound)}",
    )


def check_wallis(pi_g: mp.mpf) -> CheckResult:
    """Check pi_g/2 against standard finite Wallis bounds.

        W_N < pi/2 < W_N * (2N+1)/(2N).
    """

    terms = 100_000
    product = mp.mpf(1)
    for n in range(1, terms + 1):
        n_mp = mp.mpf(n)
        product *= (4 * n_mp * n_mp) / (4 * n_mp * n_mp - 1)

    lower = product
    upper = product * mp.mpf(2 * terms + 1) / (2 * terms)
    target = pi_g / 2
    lower_gap = target - lower
    upper_gap = upper - target
    violation = max(mp.mpf(0), -lower_gap, -upper_gap)
    roundoff = mp.power(10, -(mp.dps - 15))

    return CheckResult(
        identifier="W4",
        name="Wallis product",
        domain="Infinite products",
        method="standard two-sided finite Wallis enclosure",
        observed_metric_name="interval violation",
        observed_metric=violation,
        acceptance_bound=roundoff,
        passed=(lower_gap >= 0 and upper_gap >= 0),
        details=(
            f"terms={terms}; enclosure width={sci(upper-lower)}; "
            f"lower gap={sci(lower_gap)}; upper gap={sci(upper_gap)}"
        ),
    )


def check_stirling_robbins(pi_g: mp.mpf) -> CheckResult:
    """Check exact factorials against Robbins' sharpened Stirling bounds."""

    test_values = (10, 100, 1000, 10_000)
    minimum_margin = mp.inf
    worst_n = None
    violations: list[mp.mpf] = []

    for n in test_values:
        log_factorial = mp.fsum(mp.log(k) for k in range(1, n + 1))
        base = mp.mpf("0.5") * mp.log(2 * pi_g * n) + n * (mp.log(n) - 1)
        lower = base + mp.mpf(1) / (12 * n + 1)
        upper = base + mp.mpf(1) / (12 * n)
        margin = min(log_factorial - lower, upper - log_factorial)
        violation = max(mp.mpf(0), -margin)
        violations.append(violation)
        if margin < minimum_margin:
            minimum_margin = margin
            worst_n = n

    maximum_violation = max(violations)
    roundoff = mp.power(10, -(mp.dps - 15))

    return CheckResult(
```

```
        identifier="S5",
        name="Robbins-Stirling bounds",
        domain="Asymptotics / combinatorics",
        method="exact log-factorial sum tested inside Robbins' two-sided bounds",
        observed_metric_name="maximum bound violation",
        observed_metric=maximum_violation,
        acceptance_bound=roundoff,
        passed=maximum_violation == 0,
        details=(
            f"n={list(test_values)}; smallest positive log-margin={sci(minimum_margin)} "
            f"at n={worst_n}"
        ),
    )


def check_gamma_half(pi_g: mp.mpf) -> CheckResult:
    residual = abs(mp.gamma(mp.mpf("0.5")) ** 2 - pi_g)
    return CheckResult(
        identifier="H6",
        name="Gamma at one half",
        domain="Special functions",
        method="direct residual gamma(1/2)^2 - pi_g",
        observed_metric_name="abs(gamma(1/2)^2 - pi_g)",
        observed_metric=residual,
        acceptance_bound=mp.mpf(DIRECT_RESIDUAL_TOLERANCE_TEXT),
        passed=residual <= mp.mpf(DIRECT_RESIDUAL_TOLERANCE_TEXT),
        details="library-level cross-check; not claimed to be algorithmically independent of pi",
    )


def check_euler_identity(pi_g: mp.mpf) -> CheckResult:
    residual = abs(mp.exp(mp.j * pi_g) + 1)
    return CheckResult(
        identifier="E7",
        name="Euler identity",
        domain="Complex analysis",
        method="direct complex residual exp(i*pi_g)+1",
        observed_metric_name="abs(exp(i*pi_g)+1)",
        observed_metric=residual,
        acceptance_bound=mp.mpf(DIRECT_RESIDUAL_TOLERANCE_TEXT),
        passed=residual <= mp.mpf(DIRECT_RESIDUAL_TOLERANCE_TEXT),
        details="no classical pi appears in the tested expression",
    )


def check_reflection_formula(pi_g: mp.mpf) -> CheckResult:
    points = (mp.mpf(1) / 3, mp.mpf(2) / 5, mp.mpf(1) / 7)
    maximum_normalized_residual = mp.mpf(0)
    worst_point = None

    for z in points:
        lhs = mp.gamma(z) * mp.gamma(1 - z)
        rhs = pi_g / mp.sin(pi_g * z)
        normalized = abs(lhs - rhs) / max(mp.mpf(1), abs(lhs), abs(rhs))
        if normalized > maximum_normalized_residual:
            maximum_normalized_residual = normalized
            worst_point = z

    return CheckResult(
        identifier="R8",
        name="Euler reflection formula",
        domain="Complex analysis / special functions",
        method="direct normalized residual at three rational arguments",
        observed_metric_name="maximum normalized reflection residual",
        observed_metric=maximum_normalized_residual,
        acceptance_bound=mp.mpf(DIRECT_RESIDUAL_TOLERANCE_TEXT),
        passed=maximum_normalized_residual <= mp.mpf(DIRECT_RESIDUAL_TOLERANCE_TEXT),
        details=f"z in {{1/3, 2/5, 1/7}}; worst z={mp.nstr(worst_point, 12)}",
```

```
    )


def check_richardson_enclosure(construction: PiGConstructionResult) -> CheckResult:
    """A priori enclosure for the N-series Richardson diagonal (closes the
    'internal last-diagonal delta is a diagnostic, not a bound' gap).

    For N = 6*3^j, the exact identity Q_N := N*sin(pi/N) satisfies

        Q_N = S_s(N) + T_s(N),

    where S_s(N) = sum_{k=0}^{s} (-1)^k pi^(2k+1)/(2k+1)! * N^(-2k) is the
    exact degree-s polynomial in N^(-2) coming from the classical sine
    Taylor series (a closed-form analytic fact, not a numerical fit), and
    T_s(N) is the Taylor remainder. Since pi/N <= pi/6 < 1 and the Taylor
    terms of sin are alternating with strictly decreasing magnitude on this
    range, the alternating-series remainder theorem gives the explicit,
    unconditional bound

        |T_s(N)| <= pi^(2s+3) / (2s+3)! / N^(2s+2).

    richardson_n_series is a fixed linear combination of its n inputs, with
    weights depending only on n, not on the data; those weights are
    recovered exactly via richardson_tableau_weights. With s = n - 2, the
    degree-s polynomial part S_s is annihilated exactly by an n-node
    tableau (verified separately: feeding S_s through the tableau
    reproduces pi to working-precision round-off, no remainder-bound
    slack). The only surviving contributions to |diagonal(Q) - pi| are then:

      (a) the weighted Taylor remainders  sum_i |w_i| * |T_s(N_i)|,
      (b) the weighted propagated cubic-root bracket widths, since each
          constructed value differs from the true Q_{N_i} by at most the
          certified bracket width, and
      (c) a fixed working-precision round-off allowance.

    This bound never consults the numerical value of pi_g: it uses only
    the closed-form sine Taylor remainder theorem and the data-independent
    tableau weights.
    """

    n = construction.level_count
    s = n - 2
    if s < 1:
        raise ValueError("level_count must be at least 3 for this enclosure")

    with mp.workdps(construction.work_dps):
        weights = richardson_tableau_weights(n, construction.work_dps)

        def remainder_bound(level: int) -> mp.mpf:
            n_sides = mp.mpf(6) * mp.power(3, level)
            return mp.power(mp.pi, 2 * s + 3) / mp.factorial(2 * s + 3) / mp.power(
                n_sides, 2 * s + 2
            )

        bracket_width = construction.maximum_root_bracket_width
        roundoff = mp.power(10, -(construction.work_dps - 15))

        enclosure = roundoff
        for level in range(n):
            enclosure += abs(weights[level]) * (remainder_bound(level) + bracket_width)

    residual = abs(construction.value - mp.pi)

    return CheckResult(
        identifier="N9",
        name="Richardson tableau a priori enclosure",
        domain="Asymptotic analysis / numerical linear algebra",
        method=(
```

```
            "closed-form sine Taylor remainder bound combined with the "
            "tableau's fixed data-independent linear weights and the "
            "propagated cubic-root bracket width"
        ),
        observed_metric_name="abs(pi_g - pi) [reported for comparison; not used to derive the bound]",
        observed_metric=residual,
        acceptance_bound=enclosure,
        passed=residual <= enclosure,
        details=(
            f"s={s} (degree of exactly-cancelled polynomial part); "
            f"a priori enclosure={sci(enclosure)}"
        ),
    )


def write_csv(path: Path, checks: Iterable[CheckResult]) -> None:
    fieldnames = [
        "identifier",
        "name",
        "domain",
        "method",
        "observed_metric_name",
        "observed_metric",
        "acceptance_bound",
        "passed",
        "details",
    ]
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames)
        writer.writeheader()
        for check in checks:
            writer.writerow(
                {
                    "identifier": check.identifier,
                    "name": check.name,
                    "domain": check.domain,
                    "method": check.method,
                    "observed_metric_name": check.observed_metric_name,
                    "observed_metric": sci(check.observed_metric, 30),
                    "acceptance_bound": sci(check.acceptance_bound, 30),
                    "passed": check.passed,
                    "details": check.details,
                }
            )


def build_output_text(
    construction: PiGConstructionResult,
    checks: list[CheckResult],
    reference_error: mp.mpf,
    overall_verdict: str,
) -> str:
    lines: list[str] = []
    emit = lines.append

    emit("REFEREE-SAFE PI_G CROSS-DOMAIN CONSISTENCY SUITE")
    emit("=" * 72)
    emit(f"working precision: {construction.work_dps} decimal digits")
    emit(
        f"construction inputs: L=0,...,{construction.maximum_level} "
        f"({construction.level_count} polygon values)"
    )
    emit("construction selection: final N-series value, deterministic, no pi oracle")
    emit(f"pi_g: {dec(construction.value, 90)}")
    emit(f"reference pi: {dec(mp.pi, 90)}")
    emit(f"diagnostic abs(pi_g-pi): {sci(reference_error, 30)}")
    emit(
        "internal last-diagonal delta (diagnostic, not a rigorous error bound): "
```

```
        + sci(construction.internal_last_diagonal_delta, 30)
    )
    emit("maximum cubic root bracket width: " + sci(construction.maximum_root_bracket_width, 30))
    emit(
        "maximum cubic midpoint polynomial residual: "
        + sci(construction.maximum_root_polynomial_residual, 30)
    )
    emit("")
    emit("DIRECT CHECK RESULTS")
    emit("-" * 72)
    for check in checks:
        emit(
            f"{check.identifier} {check.name}: "
            f"observed={sci(check.observed_metric, 24)}; "
            f"bound={sci(check.acceptance_bound, 24)}; "
            f"status={'PASS' if check.passed else 'FAIL'}"
        )
        emit(f"    method: {check.method}")
        emit(f"    details: {check.details}")
    emit("")
    emit(f"checks passed: {sum(c.passed for c in checks)}/{len(checks)}")
    emit(f"overall_verdict: {overall_verdict}")
    emit("")
    emit("CLAIM BOUNDARY")
    emit("-" * 72)
    emit(
        "These are cross-domain numerical consistency checks. They do not prove pi_g=pi, "
        "and the special-function checks are not claimed to be algorithmically independent "
        "of the numerical library's implementation of pi."
    )
    return "\n".join(lines) + "\n"


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output-dir", default="pi_g_verification_output")
    parser.add_argument("--work-dps", type=int, default=DEFAULT_WORK_DPS)
    parser.add_argument("--level-count", type=int, default=DEFAULT_LEVEL_COUNT)
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    output_dir = Path(args.output_dir).resolve()
    output_dir.mkdir(parents=True, exist_ok=True)

    mp.dps = args.work_dps
    construction = compute_pi_g(level_count=args.level_count, work_dps=args.work_dps)

    # This comparison is performed only after pi_g has been constructed and fixed.
    reference_error = abs(construction.value - mp.pi)

    checks = [
        check_gaussian_integral(construction.value, args.work_dps),
        check_basel(construction.value),
        check_leibniz(construction.value),
        check_wallis(construction.value),
        check_stirling_robbins(construction.value),
        check_gamma_half(construction.value),
        check_euler_identity(construction.value),
        check_reflection_formula(construction.value),
        check_richardson_enclosure(construction),
    ]

    all_pass = all(check.passed for check in checks)
    overall_verdict = (
        "PI_G_CROSS_DOMAIN_CONSISTENCY_CHECKS_PASS"
        if all_pass
```

```
        else "PI_G_CROSS_DOMAIN_CONSISTENCY_CHECKS_FAIL"
    )

    csv_path = output_dir / "pi_g_verification_results.csv"
    output_path = output_dir / "pi_g_verification_output.txt"
    summary_path = output_dir / "pi_g_verification_summary.txt"
    manifest_path = output_dir / "pi_g_verification_manifest.json"

    write_csv(csv_path, checks)
    output_text = build_output_text(construction, checks, reference_error, overall_verdict)
    output_path.write_text(output_text, encoding="utf-8")

    summary_lines = [
        "Baker pi_g referee-safe numerical verification summary",
        "=" * 58,
        f"checks_passed: {sum(c.passed for c in checks)}/{len(checks)}",
        f"working_precision_decimal_digits: {args.work_dps}",
        f"level_count: {args.level_count}",
        f"maximum_level: {args.level_count - 1}",
        f"diagnostic_abs_pi_g_minus_pi: {sci(reference_error, 30)}",
        "internal_last_diagonal_delta: " + sci(construction.internal_last_diagonal_delta, 30),
        f"overall_verdict: {overall_verdict}",
        "claim_boundary: numerical consistency checks only; equality is not proved by this suite.",
    ]
    summary_path.write_text("\n".join(summary_lines) + "\n", encoding="utf-8")

    script_dir = Path(__file__).resolve().parent
    construction_script = script_dir / "pi_g_construction.py"
    verification_script = Path(__file__).resolve()

    manifest = {
        "title": "Referee-safe cross-domain numerical consistency checks for Baker's pi_g",
        "working_precision_decimal_digits": args.work_dps,
        "level_count": args.level_count,
        "maximum_level": args.level_count - 1,
        "pi_g_decimal": dec(construction.value, 100),
        "reference_pi_decimal": dec(mp.pi, 100),
        "diagnostic_abs_pi_g_minus_pi": sci(reference_error, 40),
        "internal_last_diagonal_delta": sci(construction.internal_last_diagonal_delta, 40),
        "maximum_root_bracket_width": sci(construction.maximum_root_bracket_width, 40),
        "maximum_root_polynomial_residual": sci(
            construction.maximum_root_polynomial_residual, 40
        ),
        "software": {
            "python": platform.python_version(),
            "mpmath": mpmath.__version__,
            "platform": platform.platform(),
        },
        "checks": [
            {
                **{
                    key: value
                    for key, value in asdict(check).items()
                    if key not in {"observed_metric", "acceptance_bound"}
                },
                "observed_metric": sci(check.observed_metric, 40),
                "acceptance_bound": sci(check.acceptance_bound, 40),
            }
            for check in checks
        ],
        "all_checks_pass": all_pass,
        "overall_verdict": overall_verdict,
        "claim_boundary": (
            "Cross-domain numerical consistency checks only. The suite does not prove pi_g=pi. "
            "Special-function checks are not asserted to be implementation-independent."
        ),
        "sha256": {
            "pi_g_construction.py": sha256_file(construction_script),
```

```
            "verify_pi_g_identities.py": sha256_file(verification_script),
            "pi_g_verification_results.csv": sha256_file(csv_path),
            "pi_g_verification_output.txt": sha256_file(output_path),
            "pi_g_verification_summary.txt": sha256_file(summary_path),
        },
    }
    manifest_path.write_text(json.dumps(manifest, indent=2), encoding="utf-8")

    print(output_text, end="")
    print(f"output_dir: {output_dir}")
    print(f"manifest_sha256: {sha256_file(manifest_path)}")

    if not all_pass:
        raise SystemExit(1)


if __name__ == "__main__":
    main()
```

C Full output log

Listing 3: Canonical verification output

```
REFEREE-SAFE PI_G CROSS-DOMAIN CONSISTENCY SUITE
========================================================================
working precision: 120 decimal digits
construction inputs: L=0,...,10 (11 polygon values)
construction selection: final N-series value, deterministic, no pi oracle
pi_g: 3.14159265358979323846264338327950288419716939937510582097494459230781640628620899600343013
reference pi: 3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803483
diagnostic abs(pi_g-pi): 2.624604694196050533234281235 45e-81
internal last-diagonal delta (diagnostic, not a rigorous error bound): 1.68888783015368936452420728675e-68
maximum cubic root bracket width: 9.96319669471211514494456752278e-101
maximum cubic midpoint polynomial residual: 1.4073500513629749373227791 8456e-100

DIRECT CHECK RESULTS
------------------------------------------------------------------------
G1 Gaussian integral: observed=3.083284322282542757142 16e-75; bound=3.092354012110273267561 8e-75; status=PASS
    method: finite quadrature, analytic two-sided tail bound, repeat-precision stability check
    details: cutoff A=13; analytic tail<=3.09235401211027327e-75; repeat-precision delta=0
B2 Basel identity: observed=7.514080526272655547837 88e-76; bound=7.544779842992032207644 11e-76; status=PASS
    method: central-binomial zeta(2) series with geometric ratio tail bound
    details: terms=120; explicit tail<=7.54477984299203221e-76
L3 Gregory-Leibniz identity: observed=3.373975626230674289979 41e-71; bound=3.381231884662237752500 77e-71;
     status=PASS
    method: Euler-transformed alternating series with explicit ratio tail bound
    details: transformed terms=230; explicit tail<=3.38123188466223775e-71
W4 Wallis product: observed=0; bound=1.0e-105; status=PASS
    method: standard two-sided finite Wallis enclosure
    details: terms=100000; enclosure width=7.85396199914311595e-6; lower gap=3.92696627342962505e-6; upper gap
     =3.9269957257134909e-6
S5 Robbins-Stirling bounds: observed=0; bound=1.0e-105; status=PASS
    method: exact log-factorial sum tested inside Robbins' two-sided bounds
    details: n=[10, 100, 1000, 10000]; smallest positive log-margin=2.7777777698412699e-15 at n=10000
H6 Gamma at one half: observed=2.624604694196050533234 28e-81; bound=1.0e-65; status=PASS
    method: direct residual gamma(1/2)^2 - pi_g
    details: library-level cross-check; not claimed to be algorithmically independent of pi
E7 Euler identity: observed=2.62460469419605053323428e-81; bound=1.0e-65; status=PASS
    method: direct complex residual exp(i*pi_g)+1
    details: no classical pi appears in the tested expression
R8 Euler reflection formula: observed=4.943233174991312699132 02e-82; bound=1.0e-65; status=PASS
    method: direct normalized residual at three rational arguments
    details: z in {1/3, 2/5, 1/7}; worst z=0.4
```

```
N9 Richardson tableau a priori enclosure: observed=2.624604694196050533234 28e-81; bound
     =1.24578953357138055219 15e-77; status=PASS
    method: closed-form sine Taylor remainder bound combined with the tableau's fixed data-independent linear
     weights and the propagated cubic-root bracket width
    details: s=9 (degree of exactly-cancelled polynomial part); a priori enclosure=1.24578953357138055e-77

checks passed: 9/9
overall_verdict: PI_G_CROSS_DOMAIN_CONSISTENCY_CHECKS_PASS

CLAIM BOUNDARY
------------------------------------------------------------------------
These are cross-domain numerical consistency checks. They do not prove pi_g=pi, and the special-function
     checks are not claimed to be algorithmically independent of the numerical library's implementation of pi.
```